Gender Bias Detection — Fine-tune `microsoft/Multilingual-MiniLM-L12-H384`

6-class subtype classification on Thai social-media text (synthesizer_v2 dataset, 24,000 samples):

- **Label 0** → `GB-ATTACK` (gender-biased attack / hate speech)
- **Label 1** → `GB-NORMATIVE` (gender-role enforcement / normative bias)
- **Label 2** → `GB-SEX` (sexualization / objectification)
- **Label 3** → `neutral` (non-gender-related content)
- **Label 4** → `meta_counter` (counter-speech / gender awareness)
- **Label 5** → `gendered_insult` (insult using gendered language but not gender-biased)

Target runtime: Google Colab T4 GPU

1. Install dependencies

In [ ]:
```
!pip install -q transformers datasets accelerate scikit-learn
```

2. Mount Google Drive & locate data

Upload `train.jsonl` (from `services/synthesizer_v2/output/`) to your Drive under
`assets/train_v2/`, then adjust `DRIVE_DATA_DIR` below.
If the Drive path doesn't exist the notebook falls back to a local relative path.

In [ ]:
```
import os

# Mount Drive (comment out if running locally)
try:
    from google.colab import drive
    drive.mount("/content/drive")
    IN_COLAB = True
except ImportError:
    IN_COLAB = False
    print("Not running in Colab — assuming data is already accessible locall

# Adjust this path to where train.jsonl lives in your Drive
DRIVE_TRAIN_FILE = "/content/drive/MyDrive/assets/train_v2/train.jsonl"
```

```
# Local fallback (relative to notebook directory)
LOCAL_TRAIN_FILE = "services/synthesizer_v2/output/train.jsonl"

TRAIN_FILE = DRIVE_TRAIN_FILE if os.path.exists(DRIVE_TRAIN_FILE) else LOCAL

print("Train file:", TRAIN_FILE)
print("Exists    :", os.path.exists(TRAIN_FILE))
```

3. Configuration

In [ ]:
```
# ── Model ──────────────────────────────────────────────────────────────
MODEL_NAME  = "microsoft/Multilingual-MiniLM-L12-H384"
OUTPUT_DIR  = "/content/drive/MyDrive/assets/outputs/minilm_gender_bias_v2"
NUM_LABELS  = 6

SUBTYPE2ID = {
    "GB-ATTACK":       0,
    "GB-NORMATIVE":    1,
    "GB-SEX":          2,
    "neutral":         3,
    "meta_counter":    4,
    "gendered_insult": 5,
}
ID2LABEL = {v: k for k, v in SUBTYPE2ID.items()}
LABEL2ID = SUBTYPE2ID

# ── Data ───────────────────────────────────────────────────────────────
MAX_LENGTH  = 128   # MiniLM works well at 128
TEST_SIZE   = 0.1   # 10 % held out for validation
RANDOM_SEED = 42

# ── Training ───────────────────────────────────────────────────────────
BATCH_SIZE   = 64    # T4 can handle 64 @ max_length=128
EPOCHS       = 3
LR           = 2e-5
WEIGHT_DECAY = 0.01
WARMUP_RATIO = 0.1
```

4. Verify GPU

In [ ]:
```
import torch

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)
if torch.cuda.is_available():
    print("GPU   :", torch.cuda.get_device_name(0))
    print("VRAM  :", round(torch.cuda.get_device_properties(0).total_memory
```

5. Load & explore data

In [ ]:
```
import json
import pandas as pd

records = []
with open(TRAIN_FILE, encoding="utf-8") as f:
    for line in f:
        line = line.strip()
        if line:
            records.append(json.loads(line))

df_raw = pd.DataFrame(records)

print(f"Total records : {len(df_raw):,}")
print(f"Columns       : {list(df_raw.columns)}")
print("\nSample rows:")
print(df_raw[["text", "label", "subtype"]].head(6).to_string())
print("\nSubtype distribution:")
print(df_raw["subtype"].value_counts().sort_index())
```

6. Build dataset

In [ ]:
```
from sklearn.model_selection import train_test_split

# Map subtype string → integer label
df = df_raw[["text", "subtype"]].copy()
df = df.dropna(subset=["text", "subtype"])
df = df[df["text"].str.strip() != ""]

df["label"] = df["subtype"].map(SUBTYPE2ID)

# Sanity check — no unmapped subtypes
unmapped = df["label"].isna().sum()
if unmapped:
    print(f"WARNING: {unmapped} rows have unknown subtype values — they will
    df = df.dropna(subset=["label"])

df["label"] = df["label"].astype(int)

print(f"Total samples : {len(df):,}")
print("\nLabel distribution:")
print(df["label"].value_counts().sort_index().rename(ID2LABEL))

train_df, val_df = train_test_split(
    df[["text", "label"]],
    test_size=TEST_SIZE,
    random_state=RANDOM_SEED,
    stratify=df["label"],
)

print(f"\nTrain: {len(train_df):,}  |  Val: {len(val_df):,}")
```

7. Tokenize

In [ ]:
```
from transformers import AutoTokenizer
from datasets import Dataset

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

def tokenize(batch):
    return tokenizer(
        batch["text"],
        truncation=True,
        padding="max_length",
        max_length=MAX_LENGTH,
    )

train_ds = Dataset.from_pandas(train_df.reset_index(drop=True))
val_ds   = Dataset.from_pandas(val_df.reset_index(drop=True))

train_ds = train_ds.map(tokenize, batched=True, batch_size=512)
val_ds   = val_ds.map(tokenize,   batched=True, batch_size=512)

train_ds.set_format(type="torch", columns=["input_ids", "attention_mask", "l
val_ds.set_format(  type="torch", columns=["input_ids", "attention_mask", "l

print("Tokenisation complete.")
print("Train features:", train_ds.features)
```

8. Load model

In [ ]:
```
from transformers import AutoModelForSequenceClassification

model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME,
    num_labels=NUM_LABELS,
    id2label=ID2LABEL,
    label2id=LABEL2ID,
)
model.to(device)

total_params = sum(p.numel() for p in model.parameters())
print(f"Total parameters: {total_params:,}")
```

9. Metrics

In [ ]:
```
import numpy as np
from sklearn.metrics import accuracy_score, f1_score, precision_score, recal

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    return {
        "accuracy" : accuracy_score(labels, preds),
        "f1"       : f1_score(labels, preds, average="macro"),
        "precision": precision_score(labels, preds, average="macro", zero_di
```

```
        "recall"   : recall_score(labels, preds, average="macro", zero_divis
    }
```

10. Training arguments

In [ ]:
```
from transformers import TrainingArguments, Trainer

steps_per_epoch = len(train_ds) // BATCH_SIZE
total_steps     = steps_per_epoch * EPOCHS
warmup_steps    = int(total_steps * WARMUP_RATIO)

training_args = TrainingArguments(
    output_dir                  = OUTPUT_DIR,
    num_train_epochs            = EPOCHS,
    per_device_train_batch_size = BATCH_SIZE,
    per_device_eval_batch_size  = BATCH_SIZE * 2,
    learning_rate               = LR,
    weight_decay                = WEIGHT_DECAY,
    warmup_steps                = warmup_steps,
    eval_strategy               = "epoch",
    save_strategy               = "epoch",
    load_best_model_at_end      = True,
    metric_for_best_model       = "f1",
    greater_is_better           = True,
    logging_steps               = 50,
    fp16                        = torch.cuda.is_available(),   # AMP on T4
    dataloader_num_workers      = 2,
    report_to                   = "none",
    save_total_limit            = 2,
)

print(f"Steps/epoch : {steps_per_epoch}")
print(f"Total steps : {total_steps}")
print(f"Warmup steps: {warmup_steps}")
```

11. Train

In [ ]:
```
trainer = Trainer(
    model           = model,
    args            = training_args,
    train_dataset   = train_ds,
    eval_dataset    = val_ds,
    compute_metrics = compute_metrics,
)

trainer.train()
```

12. Evaluate on validation set

In [ ]:
```
results = trainer.evaluate()
```

```
print("\n── Validation metrics ──────────────────────")
for k, v in results.items():
    print(f"  {k:<30} {v:.4f}" if isinstance(v, float) else f"  {k}: {v}")
```

13. Detailed classification report

In [ ]:
```
from sklearn.metrics import classification_report

preds_output = trainer.predict(val_ds)
y_pred = np.argmax(preds_output.predictions, axis=-1)
y_true = preds_output.label_ids

target_names = [ID2LABEL[i] for i in range(NUM_LABELS)]
print(classification_report(y_true, y_pred, target_names=target_names))
```

14. Save model & tokenizer

In [ ]:
```
trainer.save_model(OUTPUT_DIR)
tokenizer.save_pretrained(OUTPUT_DIR)
print(f"Model saved to: {OUTPUT_DIR}")
```

15. (Optional) Copy saved model to Google Drive

In [ ]:
```
if IN_COLAB:
    import shutil
    drive_out = "/content/drive/MyDrive/gender-bias-detection/minilm_gender_
    shutil.copytree(OUTPUT_DIR, drive_out, dirs_exist_ok=True)
    print(f"Copied to Drive: {drive_out}")
else:
    print("Skipped (not in Colab).")
```

16. Quick inference test

In [ ]:
```
from transformers import pipeline

clf = pipeline(
    "text-classification",
    model=OUTPUT_DIR,
    tokenizer=OUTPUT_DIR,
    device=0 if torch.cuda.is_available() else -1,
    truncation=True,
    max_length=MAX_LENGTH,
)

test_texts = [
    # GB-ATTACK (0)
    "ผู้หญิงพวกนี้มันโง่ ไม่ควรมีสิทธิ์ออกเสียง",
    "พวกทอมดี้มันป่วยทางจิต ไม่ปกติ",
    # GB-NORMATIVE (1)
```

```
    "ผู้หญิงควรอยู่บ้านเลี้ยงลูก ไม่ใช่ออกมาทำงาน",
    "ผู้ชายต้องเป็นช้างเท้าหน้า ห้ามอ่อนแอ",
    # GB-SEX (2)
    "ผู้หญิงหน้าตาดีแบบนี้ใช้ร่างกายขายตัวได้เลย",
    "นักแสดงหญิงคนนี้เซ็กซี่มาก ควรถ่ายแบบโป๊",
    # neutral (3)
    "ถนนติดมากเลยวันนี้ เบื่อพวกขับรถปาดหน้าจริงๆ",
    "ค่าครองชีพพุ่งสูงมาก หาเช้ากินค่ำลำบากจริง",
    # meta_counter (4)
    "ที่บอกว่าผู้หญิงขับรถแย่มันไม่จริง สถิติไม่ได้บอกแบบนั้น",
    "LGBT+ ก็มีสิทธิ์มีความสุขเหมือนกัน ไม่ใช่เรื่องผิดปกติ",
    # gendered_insult (5)
    "อีพวกนี้ทำอะไรไม่เป็นเลย ไม่รู้จักคิด",
    "มึงมันแม่งโง่จริง ทำไมถึงได้งี่เง่าขนาดนี้",
]

for text, result in zip(test_texts, clf(test_texts)):
    print(f"[{result['label']:<16}  {result['score']:.3f}]  {text[:60]}")
```

17. Batch inference on external CSV

In [ ]:

```
import torch
import pandas as pd
from transformers import pipeline
from tqdm import tqdm

# ── Config ──────────────────────────────────────────────────────────
INFERENCE_DATA_PATH = "/content/drive/MyDrive/assets/scraped_data.csv"
INFER_BATCH_SIZE    = 128   # adjust based on GPU memory

# ── Load pipeline (reuse if already loaded above) ──────────────────
device = 0 if torch.cuda.is_available() else -1

clf = pipeline(
    "text-classification",
    model=OUTPUT_DIR,
    tokenizer=OUTPUT_DIR,
    device=device,
    truncation=True,
    max_length=MAX_LENGTH,
)

# ── Load data ───────────────────────────────────────────────────────
try:
    inference_df = pd.read_csv(INFERENCE_DATA_PATH)
    print(f"Loaded {len(inference_df):,} samples from {INFERENCE_DATA_PATH}"

    if "text" not in inference_df.columns:
        raise ValueError("CSV file must contain a 'text' column.")

    texts = inference_df["text"].astype(str).tolist()

    # ── Run batch inference ─────────────────────────────────────────
    all_results = []
```

```
    for i in tqdm(range(0, len(texts), INFER_BATCH_SIZE)):
        batch = texts[i : i + INFER_BATCH_SIZE]
        all_results.extend(clf(batch))

    # ── Save results ──────────────────────────────────────────────────────
    inference_df["predicted_label"] = [res["label"] for res in all_results]
    inference_df["prediction_score"] = [res["score"] for res in all_results]
    inference_df.to_csv(INFERENCE_DATA_PATH, index=False)

    print("\nInference results (first 5 rows):")
    print(inference_df[["text", "predicted_label", "prediction_score"]].head
    print("\nPrediction label distribution:")
    print(inference_df["predicted_label"].value_counts())

except FileNotFoundError:
    print(f"File not found: {INFERENCE_DATA_PATH}")
except Exception as e:
    print(f"Error occurred: {e}")
```